

延伸思考：能否讓 Cyclic Weighting 變動態？

保留 cyclic 結構 ($i^* = (i \bmod K) + 1$)，但讓 weight 的「強度」或「相位」跟著輸入自適應。

目前的限制： $w_1 \sim w_4$ 在訓練後固定不變——BUS noise 和 CAFE noise 用同一組 weight。不同噪音環境下，各 Slice 的 bias 模式可能不同，靜態 weight 無法自適應調整。

方向 A：Input Scaling

輸入自適應的增益控制

保留 $w_1 \sim w_4$ 作為 base weight，
額外學一組 scaling factor：

$$\tilde{w}_i^{*(j)} = \alpha_i * w_i^{*(j)}$$

α_k 由輸入 pooling + FC 產生

類比：固定 EQ → 自動 EQ
額外參數：~2K (極少)

方向 B：Temporal Gate

時間軸上的選擇性加權

從 $h_i^{(j)}$ 自身產生 temporal gate：

$$g_i^{(j)} = \sigma(1 \times 1\text{-conv}(h_i^{(j)}))$$

$$y_i = \sum_s h_i^{(j)} \odot w_i * \odot g_i^{(j)}$$

w 管「整體重要性」

g 管「哪些 time step 啟用」

額外參數：1x1-conv / slice

方向 C：Phase Shift

循環起點的動態平移

Cyclic 映射加一個偏移量 Δ ：

$$i^* = ((i + \Delta) \bmod K) + 1$$

$\Delta \in \{0, \dots, K-1\}$ ，由輸入決定
(Gumbel-Softmax 使可微)

最忠於 cyclic 精神
額外參數：~K (最少)

Hong et al., CWM-TCN: Channel-Wise Weighting of Slice-based TCN for Noisy Speech Enhancement

Gating 機制：時間軸上的開關

Gate = 一個 0~1 的向量，逐 time step 決定要通過多少資訊。跟方向 A 的 scalar α 不同。

方向 A 的 α (回顧)

α 是一個 scalar (純量)

→ 整個向量統一縮放，不分 time step

方向 B 的 g (本頁重點)

g 是一個 vector (向量)，跟 h 等長

→ 每個 time step 各自有自己的開關值

數字例子：Gate 怎麼「開關」每個 time step

$h =$	0.8	0.3	0.9	0.5
	↓ 逐元素相乘			
計算	0.8×0.9	0.3×0.1	0.9×0.8	0.5×0.2
	↓			
$h \odot g =$	0.72	0.03	0.72	0.1
	保留	壓掉	保留	壓掉

白話解釋

$g = 0.9$ (大) → 那個 time step 的資訊幾乎全部通過

$g = 0.1$ (小) → 那個 time step 的資訊幾乎被擋掉

Gate 讓網路學會「在哪些時間點要聽、
哪些時間點要忽略」

方向 A 的 α ：一個 scalar，對整個向量統一縮放 (像音量旋鈕) 方向 B 的 g：一個 vector，對每個 time step 獨立控制 (像混音台的推桿)

三重 Element-wise Multiply : $h \odot w \odot g$

方向 A 是兩重 ($h \odot \text{scaled_w}$) ; 方向 B 加了第三重——gate g ，變成三個向量逐元素相乘。

方向 A (兩重)

$h \odot (a \cdot w) \rightarrow$ 先算 $a \cdot w$ ，再跟 h 相乘

方向 B (三重)

$h \odot w \odot g \rightarrow$ 三個向量都是 $\mathbb{R}^{1 \times L}$ ，逐元素相乘

完整數字例子：h, w, g 各 4 個值

	t=1	t=2	t=3	t=4
$h =$	0.8	0.3	0.9	0.5
$w =$	0.9	0.7	0.8	0.6
$g =$	0.9	0.1	0.8	0.2
計算	$0.8 \times 0.9 \times 0.9$	$0.3 \times 0.7 \times 0.1$	$0.9 \times 0.8 \times 0.8$	$0.5 \times 0.6 \times 0.2$
$h \odot w \odot g =$	0.65	0.02	0.58	0.06

分工

w 管「整體重要性」

$\rightarrow$ 這個 channel 在這條 Slice 的
整體信任度 (跟方向 A 的 cyclic weight 一樣)

g 管「哪些 time step 啟用」

$\rightarrow$ 語音段 (g 大) 保留，噪音段 (g 小) 壓掉

兩者相乘 = 同時在 channel 維度 和時間維度上做選擇性處理

t=1: $0.8 \times 0.9 \times 0.9 = 0.65 \checkmark$ 保留

t=2: $0.3 \times 0.7 \times 0.1 = 0.02 \times$ 壓掉

EEB588 — CWM-TCN 延伸：動態 Cyclic Weighting 方向 B

Gate 怎麼產生？從 h 本身用 1x1-Conv + Sigmoid

Gate 不是外部給的——它從每條 Slice 的 channel 輸出 h 自身「長出來」的。



公式：
$$g_i^{(s)} = \sigma(1 \times 1\text{-conv}(h_i^{(s)}))$$
 gate 從 h 本身產生，不需要外部資訊

為什麼用 1x1-Conv 而不是 FC？

1x1-Conv (方向 B 用的)

每個 time step 獨立做，共享同一組權重
輸入 $T=100 \rightarrow$ 輸出也是 $T=100$
 $\rightarrow$ 產出的 gate 長度跟 h 一樣
 $\rightarrow$ 可以逐 time step 控制

FC (方向 A 用的)

把整個向量壓成 K 個值
輸入 512 維 $\rightarrow$ 輸出 4 個 scalar
 $\rightarrow$ 沒有時間軸的解析度
 $\rightarrow$ 只能做全局縮放，無法逐 time step 控制

PyTorch : `self.gate_conv = nn.Conv1d(H, H, 1)` # kernel_size=1 就是 1x1-Conv

EEB588 — CWM-TCN 延伸：動態 Cyclic Weighting 方向 B

方向 B 完整流程：h → Gate → 三重 ⊙ → Σ

在原有的 $h \odot w$ 基礎上，多一步用 h 自身產生 gate g ，然後做三重 element-wise multiply。

已有元件

$h_i^{(s)}$ (Slice 輸出)
 $w_i^{(s)}$ (cyclic weight)

這兩個原本就有

★ 新增：Gate 產生

$h_i^{(s)} \rightarrow 1 \times 1\text{-Conv} \rightarrow \text{Sigmoid}$
 $\rightarrow g_i^{(s)} \in [0,1]^{1 \times L}$

每個 time step 各自
一個 gate 值

★ 修改：三重 ⊙ + Σ

$$y_i = \sum_{s=1}^S h_i^{(s)} \odot w_i^{(s)} \odot g_i^{(s)}$$

w 管整體重要性

g 管時間軸上的選擇性

→ 代入修改後的 Eq.(5)

原始 Eq.(5)：w 固定

$$y_i = \sum_{s=1}^S h_i^{(s)} \odot w_i^{(s)}$$

方向 B：加 gate g

$$y_i = \sum_{s=1}^S h_i^{(s)} \odot w_i^{(s)} \odot g_i^{(s)}$$

方向 A vs 方向 B 的差異

進階延伸

方向 A：α 是 scalar，由 Pooling+FC 從全局特徵產生 → 整條向量統一縮放

方向 B：g 是 vector，由 1×1-Conv 從局部特徵產生 → 每個 time step 獨立控制

方向 B 的表達能力更強（每個 time step 都有自己的 gate），程式且需要處理三重 ⊙ 的維度對齊。

EEB588 — CWM-TCN 延伸：動態 Cyclic Weighting 方向 B

方向 B 的程式改動 + 方向 A vs B 比較

改動量比方向 A 多——需要定義 gate 的 1×1-Conv、計算 gate、三重 ⊙。

init() 新增定義

```
# 原有（不動）
self.tcn_slices = ...
self.cyclic_w = ...

# ★ 新增（1 行）
self.gate_conv = nn.Conv1d(H, H, 1)
# 512→512, kernel=1
```

forward() 新增計算

```
# 原有：各 Slice 獨立做 TCN
h = [slice(x) for slice in self.tcn_slices]

# ★ 新增：產生 Gate (2 行)
g = [torch.sigmoid(self.gate_conv(h[s]))
      for s in range(S)] # 每條 Slice 各一個 gate

# 原有：Cyclic Weighting
for i in range(C):
    i_star = (i % K) + 1

# ★ 修改：三重 ⊙ (原本是 h*w，現在多乘 g)
y[i] = sum(h[s][i] * self.cyclic_w[i_star][s]
           * g[s][i] for s in range(S))
```

	方向 A：Input Scaling	方向 B：Temporal Gate
新增元件	Pooling + FC + Sigmoid	1×1-Conv + Sigmoid
動態來源	全局摘要 → K 個 scalar	局部特徵 → L 維 vector / slice
程式改動	~3 行 (★ 建議實作)	~8 行 (進階延伸)
表達能力	中等 (全局縮放)	更強 (逐 time step 控制)

EEB588 — CWM-TCN 延伸：動態 Cyclic Weighting 方向 B

為什麼語音增強需要時間軸選擇性？

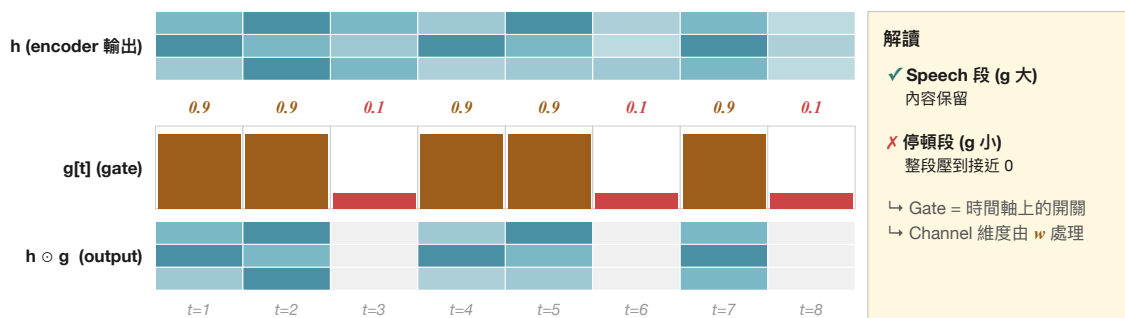
噪音不是均勻分布——語音段、停頓段、突發噪音的能量結構完全不同。



EEB588 — CWM-TCN 延伸：動態 Cyclic Weighting 方向 B

Gate 在時間軸上的開關示意

$g[t]$ 大 $\rightarrow$ 該 time step 的訊號通過； $g[t]$ 小 $\rightarrow$ 該 time step 被壓掉。



重點：Gate 不直接修改 channel 維度的內容 (那是 mask 的工作)，它只負責「逐 time step 決定通與不通」。

方向 B 用

$h \odot w \odot g$ 同時做 channel 維度 (w) 和時間維度 (g) 的選擇性。

EEB588 — CWM-TCN 延伸：動態 Cyclic Weighting 方向 B

Gating 機制的 3 個經典前身：LSTM → GRU → GLU

都是用 sigmoid 產生 0~1 的 gate 來控制資訊流——CWM-TCN 把這個想法借過來。

LSTM (1997)

Long Short-Term Memory

Hochreiter & Schmidhuber

3 個 gate

- input gate i_t
- forget gate f_t
- output gate o_t

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t])$$

作用：控制 cell state 的讀、寫、遺忘

GRU (2014)

Gated Recurrent Unit

Cho et al., EMNLP

2 個 gate

- update gate z_t
- reset gate r_t
- (沒有獨立的 cell state)

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

作用：簡化版 LSTM，少一個 gate

GLU (2017)

Gated Linear Unit (Conv)

Dauphin et al., ICML

1 個 gate

- 跟 RNN 無關
- 用 Conv 算 gate
- ← 跟方向 B 同源 ★

$$y = (Wx) \odot \sigma(Vx)$$

作用：CNN 結構也能做 gating

共通點：都用 $\sigma(\dots)$ 把實數壓到 (0,1) → gate 值；都用 $\odot$ (element-wise multiply) 把 gate 套到資訊流上。

方向 B 沿用同樣機制：

$g = \sigma(1 \times 1\text{-conv}(h))$ (與 GLU 同源，但更精簡)

EEB588 — CWM-TCN 延伸：動態 Cyclic Weighting 方向 B

1×1-Conv 的本質：每個 time step 用「同一組權重」做 FC

共享權重才能讓 gate 的長度自動跟著輸入伸縮——這是 1×1-Conv 比 FC 更適合的關鍵。

1×1-Conv 動作示意 (C = 3, T = 5)

	t=1	t=2	t=3	t=4	t=5
X	x_{11}	x_{12}	x_{13}	x_{14}	x_{15}
	x_{21}	x_{22}	x_{23}	x_{24}	x_{25}
	x_{31}	x_{32}	x_{33}	x_{34}	x_{35}

↓ W ↓ W ↓ W ↓ W ↓ W (每個 t 都用同一個 W)

	t=1	t=2	t=3	t=4	t=5
Y	y_{11}	y_{12}	y_{13}	y_{14}	y_{15}
	y_{21}	y_{22}	y_{23}	y_{24}	y_{25}
	y_{31}	y_{32}	y_{33}	y_{34}	y_{35}

公式： $y[c, t] = \sum W[c, c'] \cdot x[c', t]$ ← W 不依賴 t

1×1-Conv vs FC 對比

項目	1×1-Conv	FC
權重數量	$C \times C$ (固定)	$(C \cdot T) \times C_{out}$
time step 共享?	✓ 是	✗ 否
T 變化的適應力	T 任意	T 必須固定
Translation invar.	✓ 是	✗ 否

方向 A 用 FC (a 從全局摘要產生)；方向 B 用 1×1-Conv (gate 從局部特徵產生)。

結論： ✓ gate 跟 h 等長 ✓ 判斷邏輯跨 time step 一致 ✓ 參數少 → 訓練快、不易 overfit

EEB588 — CWM-TCN 延伸：動態 Cyclic Weighting 方向 B